Technical Due Diligence — Remediation Backlog

Platform: Forza Fantasy League (multi-sport fantasy: Football + F1 + Tennis + P2P coins) **Branch reviewed:** `v2` (incorporates all latest platform development) **Scope:** ~40K LOC React/Vite frontend · 21 Supabase Edge Functions · 229 SQL migrations · Capacitor iOS/Android **Purpose:** Internal action document. Every item below is self-contained — when you pick it up, you have the problem, the exact location, the fix, and an effort estimate so no re-discovery is needed. **Status legend:** ☐ Not started · ◐ In progress · ☑ Done

---

How to read this document

Items are ordered **in the exact sequence they should be tackled** (Section 1 = do first). Within each phase, items are ordered by dependency and risk. Each item carries:

- **ID** — stable reference (e.g. `SEC-1`). Use in commits/PRs.
- **Severity** — Critical / High / Medium / Low (acquirer's risk lens).
- **Estimate** — engineering effort for one mid/senior engineer.
- **Where** — file(s) + line(s) verified during the review.
- **Problem** — what is wrong and why it matters to a buyer.
- **Fix** — concrete remediation steps.
- **Done-when** — acceptance criteria.

Effort key: XS = <0.5 day · S = 0.5–1 day · M = 2–4 days · L = 1–2 weeks · XL = 3+ weeks

---

SEQUENCE OF WORK (at a glance)

Order	Phase	Items	Rationale
1	**Phase 0 — Pre-close security gate**	SEC-1, SEC-2, SEC-3, SEC-4, MONEY-1	Live integrity/privilege holes + revenue path. Cheap, fast, deal-blocking.
2	**Phase 1 — Stabilize foundations**	DATA-1, OPS-1, DEPLOY-1, CI-1, DEPS-1, OPS-2, CODE-1	Reproducibility, recoverability, deploy safety, visibility.
3	**Phase 2 — De-risk core logic**	TEST-1, DATA-2, DATA-3, CODE-3	Lock down money/game logic with tests; pay down data-layer debt.
4	**Phase 3 — Team-ready & scale**	CODE-2, CODE-4, CODE-5, DEPS-2, BUILD-1, INFRA-1, polish	Make the codebase safe for a team to scale and a new sport cheap to add.

Total indicative effort to "acquirer-ready": ~5–6 engineer-months, front-loaded on security (week 1) and data-layer process (weeks 2–6).

PHASE 0 — Pre-close security gate (≈1 engineer-week total)

> These are live, verified privilege-escalation / integrity holes. They are independently game-over for data integrity and must be closed before any acquirer security review or before real money flows.

SEC-1 — Admin privilege escalation: `is_admin` is client-writable 🔴 CRITICAL

- **Estimate:** S (0.5–1 day)
- **Where:** `supabase/migrations/47_rls_core_tables.sql:115-118` (the `users` UPDATE policy) + `supabase/migrations/191_f1_paddocks_schema.sql:4` (adds `is_admin boolean`). **No guard trigger on `users`** (verified — only `squads` and `coin_wallets` have column guards).
- **Problem:** The `users` UPDATE RLS policy is `USING (id = auth.uid()) WITH CHECK (id = auth.uid())` — it restricts *which row* you can edit but **not which columns**. Migration 191 added `is_admin` to that same table. Any authenticated user can run `supabase.from('users').update({ is_admin: true }).eq('id', <own uid>)` and become an admin. `is_admin = true` gates the admin-write RLS on `f1_scores` (191:153), `f1_year_results` (191:176), and `f1_races` (191:75) — so a self-promoted user can rewrite F1 scores and results. `src/screens/f1/F1AdminScreen.jsx:33` reads the same column client-side to render the admin panel.
- **Fix:** Add a `BEFORE UPDATE` trigger on `public.users` that rejects any client-initiated change to `is_admin` (and other identity/privilege columns), mirroring the existing `guard_squad_protected_columns()` pattern (migration 123). Admin assignment should only happen via a service-role RPC or the Supabase dashboard.
- **Done-when:** A simulated authenticated JWT cannot set `is_admin=true`; service-role path still can.

SEC-2 — Scoring Edge Functions have no caller authorization 🔴 CRITICAL

- **Estimate:** S (0.5–1 day)
- **Where:** `supabase/functions/score-f1-race/index.ts:96-108` (service-role client at line 104, no auth check), `score-tennis-tournament/index.ts:87-99`, `score-atp-finals/index.ts`, `sync-tennis-players/index.ts`. None are listed in `supabase/config.toml`, so they default to `verify_jwt = true` (any authenticated user passes).
- **Problem:** Each function instantiates a **service-role** Supabase client and then reads `race_id` / `tournament_id` straight from the request body with **no `requireServiceRole()` call and no identity check**. Any user with an account can POST to these endpoints and trigger a full service-role scoring write — recomputing/overwriting points and bypassing the `is_admin` RLS on the score tables entirely.
- **Fix:** Add `const authErr = await requireServiceRole(req); if (authErr) return authErr;` at the top of each handler. The correct helper already exists at `supabase/functions/_shared/auth.ts:15-58` and HMAC-verifies the token.
- **Done-when:** An authenticated (non-service-role) call to each of the four functions returns 401/403; cron/service-role calls still succeed. Redeploy each function after the fix (`npx supabase functions`

`deploy <name> --project-ref sssmvihxtqtohisghjet`).

SEC-3 — `calculate-scores` trusts an unsigned `service_role` claim 🔴 CRITICAL

- **Estimate:** S (0.5–1 day)
- **Where:** `supabase/functions/calculate-scores/index.js:203-221`; `verify_jwt = false` in `supabase/config.toml:11`.
- **Problem:** "Path B" decodes the JWT payload and trusts `payload.role === 'service_role'` **without verifying the signature** (the in-code comment at lines 204-206 explicitly acknowledges this). A forged token `{"role":"service_role"}` passes. "Path C" (lines 218-220) then lets any valid authenticated user JWT through, and the function writes `fantasy_points` DB-wide. So any logged-in user can trigger a global rescore for a fixture. This is the exact issue the project's own sale-readiness doc flagged as deal-threatening.
- **Fix:** Replace the manual claim-decode block with the HMAC-SHA256 verification from `_shared/auth.ts` (`requireServiceRole`). Remove the authenticated-user acceptance path unless there is a documented product reason (and if so, scope it to the caller's own league).
- **Done-when:** Forged/unsigned `service_role` tokens are rejected; only a genuinely signed service-role token (or the documented narrow user path) can write scores.

SEC-4 — GitHub PAT embedded in git remote URL 🔴 CRITICAL (secrets hygiene)

- **Estimate:** XS (<0.5 day) + coordination
- **Where:** `git remote get-url origin` → `https://<PAT>@github.com/...`; documented as the standard workflow in `CLAUDE.md` and `docs/superpowers/plans/*`.
- **Problem:** A live Personal Access Token granting repo write access lives in `.git/config` on developer machines and inside the synced OneDrive folder, and the docs instruct extracting it for every PR operation.
- **Fix:** Rotate the PAT immediately. Switch to SSH keys, a git credential helper, or a GitHub App. Scrub the "extract the token from the remote URL" instructions from all docs. Remove `supabase/.temp/` from git tracking and add to `.gitignore`.
- **Done-when:** No credential is stored in `.git/config`; docs no longer reference an embedded token; old PAT revoked.

MONEY-1 — `purchase-coins` revenue path is unhardened 🔴 CRITICAL (before money flows)

- **Estimate:** M (2–4 days incl. review)
- **Where:** `supabase/functions/purchase-coins/index.ts:69` (signature compare), `:23,82-116` (`MOCK_PAYMENTS`), `:26` (CORS), idempotency around `:206-215`.
- **Problem:** (a) The Stripe webhook signature check uses `if (hex !== sig) return false` — a **non-constant-time** comparison (timing-attack weakness on HMAC). (b) Idempotency relies on `SELECT … limit(1)` then insert — a TOCTOU race with no DB-level uniqueness guarantee. (c) `MOCK_PAYMENTS=true` credits coins on any authenticated request with **no Stripe charge** — if ever set in prod, free coin minting. (d) Wildcard CORS (`Access-Control-Allow-Origin: '*'`) on a

money endpoint. (e) Zero tests on the payment fulfilment path. The function is currently a skeleton (returns 503 until Stripe keys set), so it is not yet live — fix before enabling.

- **Fix:** Use a constant-time comparison for the signature; add a `UNIQUE` constraint on `coin_transactions.reference_id` to make idempotency DB-enforced; hard-fail if `MOCK_PAYMENTS=true` on the production project ref; allowlist frontend origin(s) in CORS; add integration tests for the fulfilment path; security-review before enabling Stripe.
- **Done-when:** Webhook verification is constant-time; duplicate `reference_id` insert is rejected by the DB; `MOCK_PAYMENTS` cannot be true in prod; tests cover charge→credit→idempotent-replay.

PHASE 1 — Stabilize foundations (≈3–4 weeks total)

DATA-1 — No reproducible schema; DB cannot be rebuilt from the repo 🔴 CRITICAL

- **Estimate:** L (1–2 weeks)
- **Where:** `supabase/migrations/` (229 files); CLAUDE.md: "*Supabase migration history is not used in this project*". 16 duplicate migration numbers (e.g. `159_` has 4 different files; `140_`, `141_` – `145_`, `156_`, `157_`, `187_`, `191_` collide). One-off production data fixes (e.g. `139_intfriendly_reset.sql`, `165_…stale_lock.sql`, `191_clean_sheet_retroactive_fix.sql`) are interleaved with DDL. Numbering gaps at 52, 58, 196.
- **Problem:** The migration directory is a *changelog*, not a *build script*. There is no `supabase_migrations.schema_migrations` tracking table, files were hand-applied via `npx supabase db query --linked`, and replaying `00→209` in order would error or corrupt (data fixes assume live state). An acquirer cannot stand up a clean environment from the repo.
- **Fix:** (1) Capture an authoritative `pg_dump --schema-only` from production and commit as `supabase/schema.sql` — this becomes the canonical baseline. (2) Renumber future migrations to timestamp-based unique IDs (`YYYYMMDDHHMMSS_`). (3) Separate data-fix scripts from schema DDL into a `supabase/data-fixes/` folder, excluded from the schema build. (4) Adopt the Supabase migration framework (`supabase db push`) going forward so files are the source of truth.
- **Done-when:** A fresh Supabase project can be built from `schema.sql` in one command and matches production structurally.

OPS-1 — Single environment, no PITR, manual broken backups 🔴 CRITICAL

- **Estimate:** M (2–4 days, mostly provisioning) + ongoing
- **Where:** CLAUDE.md Pilot Safeguards: live DB is the only DB; no Point-in-Time Recovery; `npx supabase db dump` "broken on this machine" (Docker unavailable); backups are hand-`SELECT`ed JSON in gitignored `backups/`.
- **Problem:** Every schema change and data fix runs directly against production with no staging rehearsal and no automated rollback. A single bad `UPDATE`/`DROP` is unrecoverable.
- **Fix:** Enable Supabase PITR (paid tier). Provision a **staging** Supabase project mirroring prod for migration rehearsal and integration tests. Automate daily off-site logical backups with a scheduled,

verified restore test. Fix the local dump tooling (or run dumps from a machine with Docker / via Supabase platform backups).
- **Done-when:** PITR enabled; a staging project exists; an automated daily backup runs and a documented restore has been successfully tested at least once.

DEPLOY-1 — Edge Function & migration deploys are manual and ungated 🟠 HIGH

- **Estimate:** M (2–4 days)
- **Where:** CLAUDE.md: Vercel auto-deploys only the React frontend; Edge Functions need manual `npx supabase functions deploy`; `migrate.yml` is `workflow_dispatch` only, default `dry_run=true`.
- **Problem:** Code can merge to `main` / `v2` (frontend auto-deploys) while the matching Edge Function binary and DB migration are NOT applied — a guaranteed frontend/backend version-skew hazard. The recurring "migration X PENDING deploy" notes in history confirm this happens (e.g. migration 209 currently committed-but-unapplied).
- **Fix:** Build a release pipeline that deploys functions + applies migrations behind a "migrations applied" gate. At minimum, add a CI check that diffs deployed function versions vs. repo and fails if drift exists.
- **Done-when:** No path exists to ship frontend code whose backend dependencies aren't deployed; a drift check runs on merge.

CI-1 — CI has no security scanning, dep audit, secret scanning, or deploy gates 🟠 HIGH

- **Estimate:** M (2–4 days)
- **Where:** `.github/workflows/ci.yml` runs only lint + build + `platform.spec`. No `npm audit`, no SAST (CodeQL/Semgrep), no secret scanning (gitleaks), no `tsc`. The `typecheck` script exists but isn't wired in.
- **Problem:** A live anon key + project ref were already committed (see INFRA-1); the absence of secret scanning is a demonstrated gap. No gate stops merging known-vulnerable deps.
- **Fix:** Add to the PR pipeline: `npm audit --audit-level=high`, CodeQL, gitleaks/trufflehog, and a **production-mode build smoke check** (catches the Rolldown TDZ class — see CODE-1). Remove or fix the dead `typecheck`.
- **Done-when:** PRs fail on high vulns, leaked secrets, and prod-build errors.

DEPS-1 — Known-vulnerable dependencies 🟠 HIGH

- **Estimate:** S (0.5–1 day) + regression test
- **Where:** `npm audit`: 8 vulns — 4 high, 3 moderate, 1 low. Notably **react-router / react-router-dom 7** (vendored turbo-stream → "unauth RCE" advisory, plus DoS/CSRF), **vite 8** (`server.fs.deny` bypass on Windows; dev-only), **ws** (memory disclosure / DoS).
- **Problem:** The react-router advisory affects a core direct dependency in a production app.
- **Fix:** Bump react-router to the fixed release; re-run `npm audit` and confirm zero high. Smoke-test routing. Make audit a CI gate (CI-1).

- **Done-when:** `npm audit` reports 0 high; routing regression suite passes.

OPS-2 — No production error tracking or alerting 🟠 HIGH

- **Estimate:** M (2–4 days)
- **Where:** No Sentry/Datadog/LogRocket/PostHog anywhere in code (only in `docs/deployment/OBSERVABILITY_STRATEGY.md`). Edge Functions log via `console.*` only. CLAUDE.md: alerting "deferred". A `cron_job_status()` RPC + admin error panel exist (migration 92) but require someone to look.
- **Problem:** A money-handling app has no way to detect errors, failed crons, or scoring failures except user reports. Migration 124 is the cautionary tale — the bet auto-resolve cron failed `UNAUTHORIZED` on *every* call and was found by manual inspection, not alerting.
- **Fix:** Integrate Sentry (frontend + Edge Functions). Add failed-cron / error-log-threshold alerting (Supabase scheduled function → email/Slack/webhook). Wire `edge_function_error_log` into the alert path.
- **Done-when:** Frontend and function errors surface in Sentry; a deliberately-failed cron triggers an alert.

CODE-1 — Rolldown (Vite 8) TDZ trap has no CI guard 🟠 HIGH

- **Estimate:** S (1–2 days)
- **Where:** CLAUDE.md documents 3 production-only crashes (`Cannot access 'X' before initialization`) from importing the same module at two depths (`useTransfer`, `BetCreatorPanel`, `HubShared`). `madge` is not installed.
- **Problem:** The bundler builds successfully and only crashes in **minified production** — dev mode masks it. The mitigation is tribal knowledge in CLAUDE.md, not a build gate. Every PR adding an import to a large screen's child tree risks a white-screen prod crash.
- **Fix:** Add `madge` to devDependencies; wire `madge --circular src/` into CI as a blocking step; add the production-mode build smoke check (CI-1). Longer-term, breaking up god-components (CODE-2) shrinks the shared-module surface that triggers this.
- **Done-when:** CI fails on circular deps and on a prod build that throws; the manual grep ritual is replaced by automation.

PHASE 2 — De-risk core logic (≈6–8 weeks total)

TEST-1 — ~0% automated coverage of money/game logic; tests run against production 🔴 CRITICAL

- **Estimate:** XL (3+ weeks for the first meaningful harness; ongoing thereafter)
- **Where:** No unit tests anywhere (no Vitest/Jest config). CI runs only `platform.spec.js` (render smoke). Logic specs (`scoring-pipeline.spec.js`, `draft-*.spec.js`, `multi-league-and-bets.spec.js`) assert on hardcoded **production** rows and are excluded from CI

(`playwright.config.js:20-31` testIgnore). Several assertions are literally `expect(body.length).toBeGreaterThan(0)`.

- **Problem:** The core financial/game logic (transfers, betting, scoring, coin ledger) has no regression protection. The 229-migration history *is* the bug log — regressions were caught by live users, not tests. Tests that touch production are non-reproducible and risky.
- **Fix:** Stand up a local/staging Postgres (`supabase start`) with deterministic seed data. Build a pgTAP or Deno integration suite covering — in priority order — the fragility-hotspot RPCs (see DATA-2): `resolve_bet` , `execute_transfer_atomic` , `set_lineup` , `place_bid` , `confirm_auction_win` , `credit_coins` / `debit_coins_to_escrow` , and the `calculate-scores` scoring math. Repoint the Playwright logic specs at seeded staging, not prod. Wire the new suite into CI.
- **Done-when:** Core RPCs have happy-path + edge-case tests running in CI against an ephemeral DB; no test reads production.

DATA-2 — Core RPCs are fragility hotspots (reactive hotfixing + drift) 🟠 HIGH

- **Estimate:** L (1–2 weeks, partly overlaps TEST-1)
- **Where:** Migration counts patching each RPC: `resolve_bet` ×14, `execute_transfer_atomic` ×12, `set_lineup` ×11, `place_bid` ×10, `accept_trade_proposal` ×8, `get_transfer_window_status` ×7, `set_captain` ×6. Migrations 133–191 are almost entirely reactive fixes. Migration 161 admits "*logic was patched into the live function ahead of this file being committed.*"
- **Problem:** The volume and total-failure nature of past bugs (e.g. mig 153: every trade acceptance errored; mig 124: bets never auto-resolved; mig 168: one live fixture locked every sub-out across all leagues) indicate the core logic was shipped without test coverage and hardened by trial-and-error in production. Because functions were hand-patched, the repo definitions may not match live.
- **Fix:** Diff every `CREATE OR REPLACE FUNCTION` in the repo against live `pg_get_functiondef` output; reconcile discrepancies into the schema baseline (DATA-1). Treat these RPCs as the highest-risk assets and front-load their test coverage (TEST-1). Budget for additional latent edge-case bugs.
- **Done-when:** Repo function bodies provably match production; the hotspot RPCs are test-covered.

DATA-3 — Coin/P2P ledger unsettled; migration 209 unapplied 🟠 HIGH

- **Estimate:** S (0.5–1 day) + compliance review (external)
- **Where:** `supabase/migrations/209_coin_ledger_compliance.sql` (header: "*DO NOT APPLY from this machine*"). MEMORY notes it's not applied. It corrects `coin_transactions.currency` defaulting to `'GBP'` (a real ISO-4217 code) for what is an internal virtual token, and adds `wager_*` type aliases.
- **Problem:** The newest, least-settled subsystem touches money. An unapplied compliance migration means the live ledger may still label virtual tokens as GBP currency — a diligence/compliance red flag for anything wager-adjacent.
- **Fix:** Confirm the live state of `coin_transactions` ; apply 209 in a controlled window (from the linked machine); obtain legal/compliance sign-off on the virtual-token accounting model before shipping real-money features.

- **Done-when:** 209 applied; ledger no longer represents tokens as fiat currency; compliance sign-off recorded.

CODE-3 — No data-fetching layer (raw Supabase scattered) 🟡 MEDIUM

- **Estimate:** L→XL (3–5 weeks to migrate)
- **Where:** `supabase.from(` in 24 files (13 screens); `supabase.rpc(` in 30. No TanStack Query/SWR in `package.json`.
- **Problem:** Every component hand-rolls loading/error/refetch with `useState` + `useEffect`. No caching, deduplication, or consistent error contract — the root cause of state-density in god-components and inconsistent error handling.
- **Fix:** Introduce TanStack Query (or a thin typed data-access module per table). Migrate fetches into it incrementally. Add an ESLint rule banning `supabase.from` inside screen components.
- **Done-when:** Screens consume data via hooks/query; no raw `supabase.from` in screen files; loading/error states are uniform.

PHASE 3 — Team-ready & scale (≈8–12 weeks total)

CODE-2 — God-components 🟠 HIGH

- **Estimate:** XL (6–10 engineer-weeks across top 5 files)
- **Where:** `SquadScreen.jsx` (2,879 lines, 30 `useState`, 275 inline styles), `CommissionerPanel.jsx` (2,034), `LeagueScreen.jsx` (1,894, 46 `useState`), `MarketScreen.jsx` (1,530), `LiveScreen.jsx` (1,362).
- **Problem:** Data fetching + business rules + 500-line JSX interleaved in single files; effectively untestable state space; a new hire needs 4–6 weeks before they can safely change these high-traffic surfaces.
- **Fix:** Extract data/business logic into hooks (the pattern exists — `useTransfer`, `useSquad` — but is under-used); split desktop/mobile/modal render trees into child components; lift inline config out. Do incrementally **behind the TEST-1 harness**.
- **Done-when:** No screen file exceeds ~600 lines; business logic lives in tested hooks; new-engineer onboarding to these surfaces drops to ~1 week.

CODE-4 — Multi-sport is copy-paste, not abstraction 🟡 MEDIUM

- **Estimate:** L (2–4 weeks)
- **Where:** `src/screens/f1/` (7 screens) and `src/screens/tennis/` (7 screens) are fully separate trees; `SportContext.jsx` is just 3 localStorage IDs.
- **Problem:** The platform's headline selling point (multi-sport extensibility) currently costs a full 7-screen clone per new sport. Cross-cutting concerns (leaderboard, standings, admin result entry, scoring/gazette plumbing) are duplicated, not shared.
- **Fix:** Extract shared sport-agnostic primitives (leaderboard table, standings, admin-result form, result card) parametrized by a sport config object. Keep genuinely sport-specific UIs separate.

- **Done-when:** Adding a new sport reuses shared primitives; only sport-specific screens are net-new.

CODE-5 — `typecheck` is dead; codebase 100% untyped 🟡 MEDIUM

- **Estimate:** XS to remove the dead script; XL for meaningful TS adoption (strategic, multi-month)
- **Where:** `package.json` `"typecheck": "tsc --noEmit"` but no `tsconfig.json`, no TS dep, zero `.ts` / `.tsx` in `src/`.
- **Problem:** A `typecheck` script implies a safety net that doesn't exist. Untyped DB rows + untyped props make refactoring high-risk (no compiler catches a renamed column).
- **Fix:** Remove the dead script now (5 min). Strategically, adopt incremental TypeScript from `lib/` and `hooks/` outward; generate Supabase types (`supabase gen types`).
- **Done-when (near-term):** No misleading dead script. **(strategic):** core `lib` / `hooks` typed; DB types generated.

DEPS-2 — Bleeding-edge dependency stack 🟡 MEDIUM

- **Estimate:** S (policy) + ongoing
- **Where:** Vite 8/Rolldown, React 19, react-router 7, TypeScript 6, Tailwind 4, Capacitor 8; CI Node 24 (not LTS-pinned).
- **Problem:** Several were recent/early-stage; elevated upgrade churn. The Rolldown TDZ class (CODE-1) is a direct symptom.
- **Fix:** Document a dependency-stability policy; pin to LTS Node; treat the prod-build smoke check (CI-1) as a standing requirement.
- **Done-when:** Stability policy documented; Node version pinned (see BUILD-1).

BUILD-1 — Non-deterministic build; Node version unpinned 🟡 MEDIUM

- **Estimate:** XS (<0.5 day)
- **Where:** No `.nvmrc` / `.node-version`, no `engines` field. Mobile workflow uses `npm install` (not `npm ci`). `package-lock.json` present (good).
- **Problem:** A new owner has no signal for the required Node version; native installs aren't reproducible.
- **Fix:** Add `engines.node` + `.nvmrc`; standardize on `npm ci` everywhere; add the prod-mode build smoke check.
- **Done-when:** Node pinned; all CI uses `npm ci`; build verified in prod mode.

INFRA-1 — Owner-tied infrastructure 🟡 MEDIUM

- **Estimate:** M (2–4 days at transfer time)
- **Where:** Project ref `sssmvihxtqtohisghjet` hardcoded in ~42 files incl. cron URLs baked into migrations (60, 63, 90, 91, 108, 110, 120, 122, 127, 181); `supabase/.temp/` committed.
- **Problem:** Ownership transfer requires find/replace across 40+ files (including append-only migrations) and re-keying all secrets.
- **Fix:** Externalize the project ref to config/env. Remove `supabase/.temp/` from git. Template cron URLs. Document the full transfer runbook (new Supabase + Vercel + GitHub +

Forza/Stripe/Groq/RapidAPI accounts + key rotation).
- **Done-when:** No owner-specific ref hardcoded in source; a transfer runbook exists and has been dry-run.

LOW / POLISH (tackle opportunistically)

ID	Item	Est	Note
LOW-1	No route-level code-splitting (`React.lazy` / `Suspense` = 0 in `App.jsx`)	S	Whole app incl. admin/F1/tennis in initial bundle; lazy-loading also mitigates CODE-1.
LOW-2	Wildcard CORS on money endpoint; `player_boxes` / `match_events` broad `USING (true)` reads	S	Verify no PII exposed; allowlist origins.
LOW-3	~3,795 inline `style={}` blocks bypassing Tailwind/Kit-Light tokens	L	Extract repeated style objects to shared constants; incremental.
LOW-4	Shipped feature-flag dead code (`CHIPS_ENABLED`, `KNOCKOUT_DRAFT_ENABLED`, inert wildcard columns)	S	Tree-shake disabled features or move to a branch.
LOW-5	Source-file encoding mojibake (Windows UTF-8) — has caused real SQL bugs	XS	Add `.editorconfig` (UTF-8 + LF) + CI non-UTF-8 byte check.
LOW-6	External-API SPOF: Forza (sole football feed); RapidAPI tennis on **50 req/day free tier**, no retry/backoff	M	Confirm commercial SLAs; move tennis to paid plan before scale; add backoff + stale-data alarm.
LOW-7	Recovery tables (`round_backups`, `squad_events`) cover game-state only, not full-DB	—	Adequate for game state; not a substitute for PITR (OPS-1).
LOW-8	Run transitive license scan (`license-checker --summary`) for the data room	XS	Direct deps are clean (MIT/Apache/ISC/BSD); confirm transitive.
LOW-9	Mobile (Capacitor) not store-ready: simulator/debug only, unsigned, no certs/listings	L+	Treat as roadmap, not a shipped asset.

What is genuinely solid (credit where due — useful for the buyer narrative)

- `_shared/auth.ts` `requireServiceRole()` correctly HMAC-verifies — the gold-standard pattern; the fix for SEC-2/SEC-3 is to *apply it*, not invent it.
- `purchase-coins` webhook sources price/coins server-side, takes `user_id` from the authenticated JWT (not client body), and is idempotent on `reference_id` (needs a DB constraint to be bulletproof — MONEY-1).

- Coin ledger (`coin_wallets` / `coin_transactions`) has RLS + a guard trigger; `credit_coins` / `debit_coins_to_escrow` / `release_escrow` / `admin_grant_coins` are REVOKE'd from anon/authenticated.
- `squads` column guard (`guard_squad_protected_columns`) is intact — clients cannot write `budget_remaining` /roster.
- `AuthContext` uses `VITE_AUTH_ENABLED === 'true'` only (the `|| import.meta.env.PROD` bypass was removed). No demo-mode auth bypass.
- No `dangerouslySetInnerHTML` / `eval` in `src/` ; chat renders as escaped JSX.
- Scoring pipeline is **idempotent** (upserts keyed on `fixture_id,player_id` and `squad_id,matchday_id`) with redundant overlapping cron windows and a settled-round guard — genuinely well-designed; it just lacks alerting (OPS-2).
- Per-route `ErrorBoundary` with a client crash reporter limits blast radius — better than most codebases this size.
- Audit/recovery tables (`round_backups` , `squad_events` , `squad_matchday_snapshots`) provide a solid game-state recovery story.

Last Updated: 2026-06-26
